



Lista prática 1

Computação Evolucionária

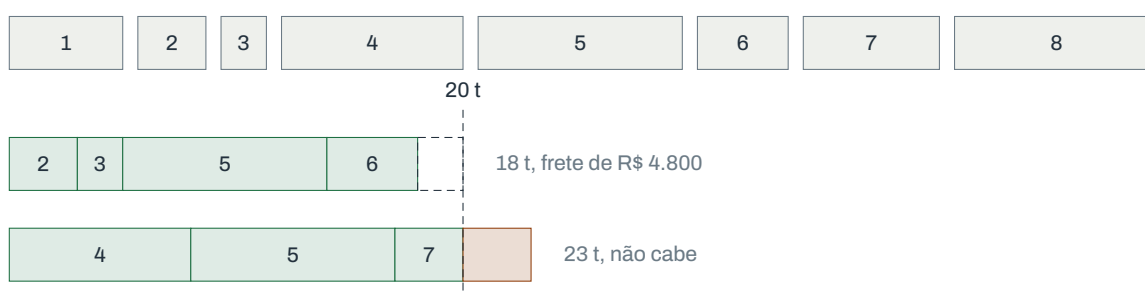
Prof. Ygor Amaral B. L de Sena <ygor.amaral@ufrpe.br>

Engenharia de Computação • UABJ/UFRPE

1 Mochila 0/1

0/1 knapsack problem • otimização restrita (COP)

Qual carga vale mais? O caminhão faz uma viagem só e leva no máximo 20 toneladas. No galpão esperam oito lotes que somam 46 toneladas, mais do que o dobro do que cabe. Cada lote paga um frete. O que fica para trás não paga nada.



Os oito lotes, com a largura proporcional ao peso, mais duas cargas montadas com eles.

A instância: Oito lotes, com o peso em toneladas e o frete em reais. O caminhão leva 20 toneladas por viagem.

Lote	1	2	3	4	5	6	7	8
Peso (t)	5	3	2	8	9	4	6	9
Frete (R\$)	500	800	900	1900	1700	1400	1300	1300

O que é uma solução: Uma decisão de levar ou deixar, para cada um dos oito lotes. Ela é válida quando o peso somado dos lotes levados não passa de 20 toneladas. O lote vai inteiro ou não vai: não existe meia carga.

Como é o indivíduo: Uma lista de oito casas, uma por lote, com 1 para levar e 0 para deixar. A lista [0, 1, 1, 0, 0, 1, 0, 0] leva os lotes 2, 3 e 6. Ela pesa $3 + 2 + 4 = 9$ toneladas e arrecada R\$ 3.100. A população inicial é um punhado de listas assim.

Objetivo e restrições: Maximizar o frete arrecadado, sem passar das 20 toneladas do caminhão. Um objetivo e uma restrição: otimização restrita.

Como se mede: A qualidade de uma carga válida é o frete que ela arrecada, a soma dos fretes dos lotes levados. Quanto maior, melhor. Uma carga que passa das 20 toneladas não pode ser executada. Decidir o que fazer com ela faz parte do seu trabalho.

O alvo: A melhor carga possível arrecada **R\$ 5.500**, pesando 20 toneladas.

Entrada e saída: Os dados da tabela entram no seu programa como você preferir, escritos no código ou lidos de um arquivo. Na saída, imprima a carga encontrada, o peso dela, o frete dela e

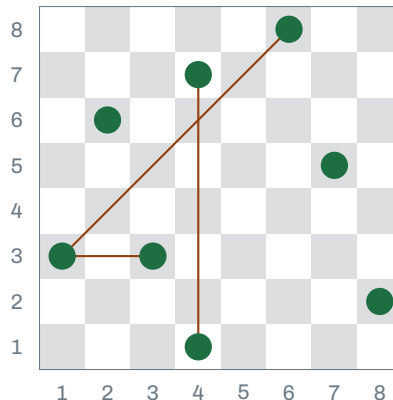
a diferença para o alvo.

O que entregar: A modelagem escrita e o programa rodando sobre a instância acima.

2 N-rainhas

N-queens problem • satisfação de restrições (CSP)

Onde cabem oito rainhas sem que nenhuma ataque outra? A rainha alcança tudo o que estiver na sua linha, na sua coluna ou nas suas diagonais. O tabuleiro tem 64 casas para as oito peças.



Oito rainhas dispostas ao acaso, com três pares em conflito: um divide a linha, um divide a coluna e um divide a diagonal.

A instância: Um tabuleiro de oito por oito e oito rainhas. Não há mais dado nenhum.

O que é uma solução: Uma disposição das oito rainhas no tabuleiro, uma casa para cada. Aqui nenhuma disposição é proibida de existir: o que muda de uma para outra é quantos ataques ela tem.

Como é o indivíduo: Uma permutação das oito linhas, com uma rainha por coluna: a posição na lista é a coluna e o valor guardado nela é a linha. A lista $[6, 1, 7, 5, 2, 8, 4, 3]$ põe a rainha da coluna 1 na linha 6 e a da coluna 2 na linha 1. Como nenhuma posição se repete e nenhum valor se repete, os ataques de linha e de coluna somem por construção. Sobra para a busca apenas a diagonal. Essa lista tem exatamente três pares em conflito, todos na diagonal: use-a para conferir a sua contagem antes de pôr a busca para rodar. A população inicial é um punhado de permutações assim.

Repare no que essa escolha faz: ela não alcança todas as disposições do tabuleiro. O próprio tabuleiro desenhado acima é uma delas, porque tem duas rainhas na coluna 4 e nenhuma na coluna 5: não existe permutação que o descreva. Nenhuma disposição desse tipo resolveria o problema, então o que a representação descarta é só trabalho perdido.

Objetivo e restrições: Aqui só há restrição, que é nenhuma rainha atacar outra. Entre os tabuleiros que a cumprem nenhum vale mais que os demais: satisfação de restrições.

Como se mede: Conte os pares de rainhas que se atacam. Duas rainhas se atacam quando dividem a linha, a coluna ou uma diagonal. Quanto menos pares, melhor. O problema só está resolvido quando a contagem chega a zero. É por isso que aqui não há qualidade a maximizar: há uma exigência a cumprir.

O alvo: A busca só termina com **zero conflito** no tabuleiro.

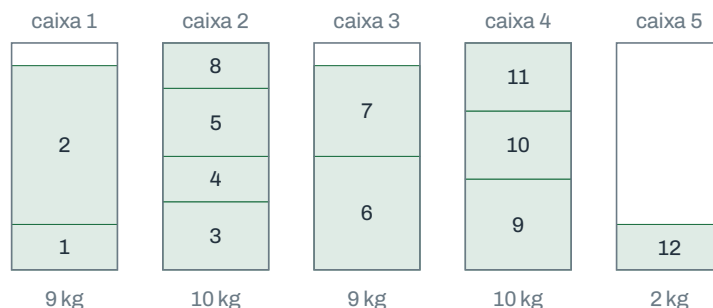
Entrada e saída: O único dado é o tamanho do tabuleiro. Na saída, imprima a posição das oito rainhas, a contagem de conflitos e o tabuleiro desenhado em texto.

O que entregar: A modelagem escrita e o programa rodando sobre o tabuleiro de oito por oito.

3 Empacotamento

bin packing problem • otimização restrita (COP)

Quantas caixas bastam? Doze pacotes saem hoje para a transportadora. Cada caixa aguenta 10 quilos e a cobrança é por caixa despachada, não por peso.



Uma arrumação possível dos doze pacotes, em cinco caixas de 10 quilos. A altura de cada pacote é o peso dele.

A instância: Doze pacotes, com o peso em quilos. Toda caixa aguenta 10 quilos e há caixas à vontade.

Pacote	1	2	3	4	5	6	7	8	9	10	11	12
Peso (kg)	2	7	3	2	3	5	4	2	4	3	3	2

O que é uma solução: Uma distribuição dos doze pacotes entre as caixas. Ela é válida quando nenhuma caixa passa de 10 quilos. Todo pacote entra em alguma caixa: nenhum fica de fora.

Como é o indivíduo: Uma lista de doze casas, uma por pacote, com o número da caixa em que ele viaja. A lista [1, 1, 2, 2, 2, 3, 3, 2, 4, 4, 4, 5] é a arrumação desenhada acima: o pacote 8 viaja na caixa 2, junto dos pacotes 3, 4 e 5. A população inicial é um punhado de listas assim.

Objetivo e restrições: Minimizar o número de caixas despachadas, sem que nenhuma passe de 10 quilos. Um objetivo e uma restrição: otimização restrita.

Como se mede: Conta-se quantas caixas a arrumação usa. Quanto menos, melhor. Os doze pacotes somam 40 quilos e cada caixa leva 10, então nenhuma arrumação usa menos de quatro caixas. A soma dividida pela capacidade é o piso de qualquer instância: ela diz o quanto ainda falta.

O alvo: A melhor arrumação possível usa **quatro caixas**, o mesmo número do piso.

Entrada e saída: Os pesos entram no seu programa como você preferir. Na saída, imprima quantas caixas a arrumação usa, o conteúdo de cada caixa e o peso que cada uma leva.

O que entregar: A modelagem escrita e o programa rodando sobre a instância acima.